

Actúa como un agente de programación experto en ESP32, MicroPython, BLE, WiFi, uasyncio, arquitectura de datos y diseño de firmware robusto para IoT. Tu tarea es DISEÑAR e IMPLEMENTAR desde cero el firmware de la aplicación "Dosimat IoT", siguiendo la arquitectura de comunicación y datos validada en RiegoSmart (podés ver el proyecto CONTROL_RIEGO).

El objetivo es crear un firmware extremadamente estable, escalable y seguro, capaz de operar en entornos reales con múltiples dispositivos, múltiples usuarios y sincronización con Firebase.

OBJETIVO GENERAL

Diseñar desde cero el firmware de Dosimat IoT con:

- Comunicación BLE minimalista
- Comunicación WiFi + MQTT robusta
- Exclusión mutua BLE/WiFi (nunca simultáneos)
- Máquina de estados de red
- Arquitectura de datos basada en IDs cortos
- Firebase como fuente de verdad (SSOT)
- Historial optimizado (buffer circular + append-only)
- Sincronización móvil ↔ Firebase ↔ ESP32
- Multi-equipos, multi-usuario y roles (propietario, técnico, super admin)

ARQUITECTURA MULTI-EQUIPO Y MULTI-USUARIO

El sistema debe soportar:

- Muchos equipos Dosimat IoT, cada uno identificado por su MAC/ID único

- Usuarios propietarios (cada uno puede tener uno o varios equipos)
- Usuarios técnicos autorizados por el super administrador
- Un super administrador con acceso global
- Control de permisos desde Firebase (no desde el ESP32)

Estructura Firebase recomendada:

usuarios/

{USER_ID}/

perfil/

equipos_asignados/

{CHIPID}: true

equipos/

{CHIPID}/

config/

nombres_tanques/

perfiles_dosis/

logs/

estado/

propietarios/

{USER_ID}: true

tecnicos_autorizados/

{USER_ID}: true

roles/

super_admin/

{USER_ID}: true

El ESP32 NO maneja usuarios.

Solo reporta su CHIPID y estado.

ARQUITECTURA DE COMUNICACIÓN

BLE minimalista

El ESP32 solo transmite:

- IDs cortos (T1, T2... P1, P2...)
- Estado del equipo
- Tiempos restantes
- Comandos simples

Nunca transmite:

- nombres largos
- configuraciones extensas
- perfiles completos

WiFi + MQTT robusto

- Conexión no bloqueante
- Reconexión automática
- Timeout TCP reducido
- Loop MQTT no bloqueante
- Publicación de telemetría compacta

Exclusión mutua BLE/WiFi

- Si no hay WiFi → BLE ON, WiFi OFF
- Si se configura WiFi → BLE OFF, WiFi ON
- Si WiFi falla → BLE ON, WiFi OFF
- Si WiFi reconecta → BLE OFF, WiFi ON

BLE y WiFi ****nunca**** deben estar activos simultáneamente.

✅ Máquina de estados BLE/WiFi

Estados:

- INIT
- BLE_ONLY
- WIFI_CONNECTING
- WIFI_ONLINE
- FALLBACK_BLE

📌 ARQUITECTURA DE DATOS

✅ IDs cortos en el ESP32

Ejemplo:

- PR1 =Programa1
- P1 = Perfil 1

✅ Nombres largos en Firebase

Ejemplo:

- “P1” → “Perfil semanal invierno”

✅ Configuración completa en Firebase

El ESP32 solo guarda:

- config_version
- parámetros mínimos
- perfiles compactados

✅ Historial optimizado

- Buffer circular en RAM
- Archivo append-only rotado por tamaño

- Subida a Firebase solo si hay WiFi

TAREAS DEL AGENTE

1. Diseñar la arquitectura completa del firmware.
2. Definir módulos y funciones.
3. Definir estructura de datos mínima local.
4. Definir estructura de datos completa en Firebase.
5. Diseñar la máquina de estados BLE/WiFi.
6. Diseñar BLE minimalista.
7. Diseñar WiFi robusto.
8. Diseñar MQTT no bloqueante.
9. Diseñar sincronización con Firebase (vía MQTT + Cloud Functions).
10. Diseñar historial optimizado.
11. Generar código de ejemplo en MicroPython.
12. Diseñar soporte multi-equipo y multi-usuario.
13. Proponer plan de migración futura (IDs híbridos, versionado).

ENTREGABLES ESPERADOS

- Arquitectura completa del firmware
- Diagnóstico de riesgos
- Módulos y funciones recomendadas
- Máquina de estados BLE/WiFi
- Diagrama de flujo
- Diseño de BLE minimalista
- Diseño de